

HW4 B063012054

two layers :

```
# Initialize parameters dictionary, by calling one of the functions you'd previously implemented
### START CODE HERE ### (~ 1 line of code)
parameters = initialize_parameters(n_x,n_h,n_y)
### END CODE HERE ###

for i in range(0, num_iterations):

    # Forward propagation: LINEAR -> RELU -> LINEAR -> SIGMOID. Inputs: "X, W1, b1". Output: "A1, cache1, A2, cache2".
    ### START CODE HERE ### (~ 2 lines of code)
    A1, cache1 = linear_activation_forward(X, W1, b1,"relu")
    A2, cache2 = linear_activation_forward(A1, W2, b2,"sigmoid")
    ### END CODE HERE ###

    # Compute cost
    ### START CODE HERE ### (~ 1 line of code)
    cost = compute_cost(A2,Y)
    ### END CODE HERE ###

    # Initializing backward propagation
    dA2 = - (np.divide(Y, A2) - np.divide(1 - Y, 1 - A2))

    # Backward propagation. Inputs: "dA2, cache2, cache1". Outputs: "dA1, dW2, db2; also dA0 (not used), dW1, db1".
    ### START CODE HERE ### (~ 2 lines of code)
    dA1, dW2, db2 = linear_activation_backward(dA2, cache2,"sigmoid")
    dA0, dW1, db1 = linear_activation_backward(dA1, cache1,"relu")
    ### END CODE HERE ###

    # Set grads['dW1'] to dW1, grads['db1'] to db1, grads['dW2'] to dW2, grads['db2'] to db2
    grads['dW1'] = dW1
    grads['db1'] = db1
    grads['dW2'] = dW2
    grads['db2'] = db2

    # Update parameters.
    ### START CODE HERE ### (approx. 1 line of code)
    parameters = update_parameters(parameters,grads,learning_rate)
    ### END CODE HERE ###
```

第一層為 Linear->relu，第二層為 Linear->sigmoid，cache 用以紀錄每一層參數變化，若是 backward 順序要相反。 ，
從趨勢圖可以發現，隨著次數增加，cost 能一直降低，但 cost 不是愈低愈好，因為這可能會有 overfitting 的問題。

```
In [11]: predictions_train = predict(train_x, train_y, parameters)
```

Accuracy: 0.9999999999999998

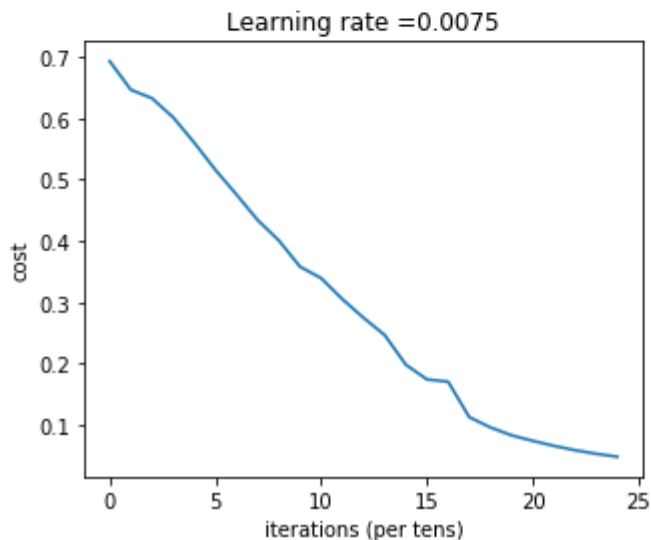
```
In [12]: predictions_test = predict(test_x, test_y, parameters)
```

Accuracy: 0.72

這是此模型訓練與測試資料的正確率，明顯是 overfitting。

結果：

```
Cost after iteration 0: 0.693049735659989
Cost after iteration 100: 0.6464320953428849
Cost after iteration 200: 0.6325140647912678
Cost after iteration 300: 0.6015024920354665
Cost after iteration 400: 0.5601966311605748
Cost after iteration 500: 0.5158304772764729
Cost after iteration 600: 0.4754901313943325
Cost after iteration 700: 0.433916315122575
Cost after iteration 800: 0.400797753620389
Cost after iteration 900: 0.35807050113237976
Cost after iteration 1000: 0.33942815383664116
Cost after iteration 1100: 0.3052753636196264
Cost after iteration 1200: 0.27491377282130147
Cost after iteration 1300: 0.24681768210614832
Cost after iteration 1400: 0.19850735037466108
Cost after iteration 1500: 0.17448318112556552
Cost after iteration 1600: 0.17080762978096664
Cost after iteration 1700: 0.11306524562164713
Cost after iteration 1800: 0.09629426845937154
Cost after iteration 1900: 0.08342617959726867
Cost after iteration 2000: 0.0743907870431908
Cost after iteration 2100: 0.06630748132267929
Cost after iteration 2200: 0.05919329501038175
Cost after iteration 2300: 0.05336140348560553
Cost after iteration 2400: 0.04855478562877014
```



N-layer

```
##### START CODE HERE #####
parameters = initialize_parameters_deep(layers_dims)
#### END CODE HERE ####

# Loop (gradient descent)
for i in range(0, num_iterations):

    # Forward propagation: [LINEAR -> RELU]*(L-1) -> LINEAR -> SIGMOID.
    #### START CODE HERE #### (~ 1 line of code)
    AL, caches = L_model_forward(X, parameters)
    #### END CODE HERE ####

    # Compute cost.
    #### START CODE HERE #### (~ 1 line of code)
    cost = compute_cost(AL, Y)
    #### END CODE HERE ####

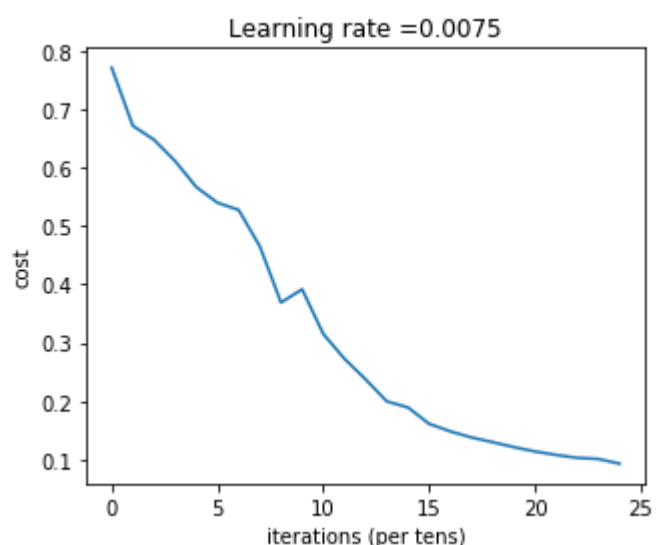
    # Backward propagation.
    #### START CODE HERE #### (~ 1 line of code)
    grads = L_model_backward(AL, Y, caches)
    #### END CODE HERE ####

    # Update parameters.
    #### START CODE HERE #### (~ 1 line of code)
    parameters = update_parameters(parameters, grads, learning_rate)
    #### END CODE HERE ####
```

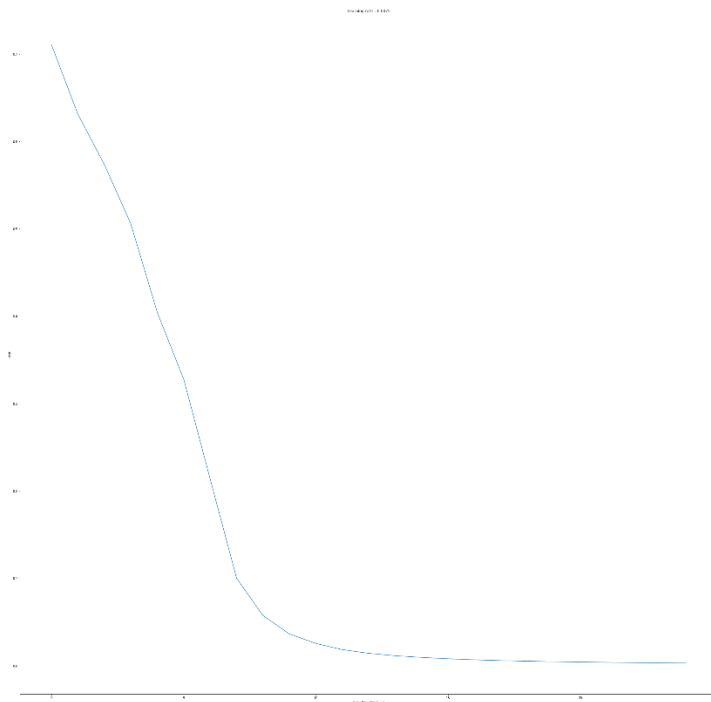
layer 數上升，增加神經節點數，降低 overfitting 的情況，從五層 layer 的模型可以看到訓練資料正確率比兩層的模型低，但測試資料正確率反而上升，減少了 overfitting 的現象，提高未知資料辨識率才是我們想要的。若想要提高正確率，可以嘗試改變起始點、學習次數、學習效率、更換激活函數或者將模型層數做改變。

結果：

```
Cost after iteration 0: 0.771749
Cost after iteration 100: 0.672053
Cost after iteration 200: 0.648263
Cost after iteration 300: 0.611507
Cost after iteration 400: 0.567047
Cost after iteration 500: 0.540138
Cost after iteration 600: 0.527930
Cost after iteration 700: 0.465477
Cost after iteration 800: 0.369126
Cost after iteration 900: 0.391747
Cost after iteration 1000: 0.315187
Cost after iteration 1100: 0.272700
Cost after iteration 1200: 0.237419
Cost after iteration 1300: 0.199601
Cost after iteration 1400: 0.189263
Cost after iteration 1500: 0.161189
Cost after iteration 1600: 0.148214
Cost after iteration 1700: 0.137775
Cost after iteration 1800: 0.129740
Cost after iteration 1900: 0.121225
Cost after iteration 2000: 0.113821
Cost after iteration 2100: 0.107839
Cost after iteration 2200: 0.102855
Cost after iteration 2300: 0.100897
Cost after iteration 2400: 0.092878
```



試著將 layer 更改成 `layers_dims = [12288, 20, 9, 7, 5, 1]`，增加一層以及結點，但卻出現 **overfitting**，從趨勢圖可以看到學習次數在 1000 次後 cost 收斂，變化趨緩。



```
In [24]: pred_train = predict(train_x, train_y, parameters)
```

Accuracy: 0.9999999999999998

```
In [25]: pred_test = predict(test_x, test_y, parameters)
```

Accuracy: 0.76

更改 iteration 成 1000，正確率下降至 0.74，因此我認為 layer 數過多使效果下降，將 layer 改為 `layers_dims = [12288, 20, 7, 5, 1]`，並將 iteration 增加到 1500，正確率確實提高，增加至 82%。

```
In [42]: pred_train = predict(train_x, train_y, parameters)
```

Accuracy: 0.9808612440191385

```
In [43]: pred_test = predict(test_x, test_y, parameters)
```

Accuracy: 0.8200000000000001

用自己圖片的測試顯示 `scipy` 找不到 `imread`，由於 `scipy` 在 1.2.0 版本後就將 `imread` 移除，但我降 `scipy` 版本到 1.1.0，`pillow` 7.0.0 版本依然不能使用。

```
-----  
AttributeError                                Traceback (most recent call last)  
<ipython-input-75-1661b83c4d0d> in <module>  
      7 #image=cv2.imread(fname)  
      8 #my_image=cv2.resize(image,(num_px*num_px*3,1))  
----> 9 image = np.array(ndimage.imread(fname, flatten=False))  
     10 my_image = scipy.misc.imresize(image, size=(num_px,num_px)).reshape((num_px*num_px*3,1))  
     11 my_predicted_image = predict(my_image, my_label_y, parameters)  
  
AttributeError: module 'scipy.ndimage' has no attribute 'imread'
```